

UNIT – 3

LEETCODE EXERCISE

1. Combine Two Tables

Table: Person

Column Name	Type
PersonId	int
FirstName	varchar
LastName	varchar

PersonId is the primary key column for this table.

Table: Address

Column Name	Type
AddressId	int
PersonId	int
City	varchar
State	varchar

AddressId is the primary key column for this table.

Problem Summary

Write a SQL query for a report that provides the following information for each person in the Person table, regardless if there is an address for each of those people:

FirstName, LastName, City, State.

SQL Schema

```
CREATE TABLE Person (personId INT PRIMARY KEY,lastName
VARCHAR(255),firstName VARCHAR(255));
CREATE TABLE Address (addressId INT PRIMARY KEY,personId INT,city
VARCHAR(255),state VARCHAR(255));
INSERT INTO Person (personId, lastName, firstName) VALUES
(1, 'Wang', 'Allen'),
(2, 'Alice', 'Bob');
INSERT INTO Address (addressId, personId, city, state) VALUES
(1, 2, 'New York City', 'New York'), (2,
3, 'Leetcode', 'California'); Person
```

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

Address

addressId	personId	city	state
1	2	New York City	New York
2	3	Leetcode	California

MySQL Solution

```
</> Code
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT
3     p.firstName,
4     p.lastName,
5     a.city,
6     a.state
7 FROM Person p
8 LEFT JOIN Address a
9     ON p.personId = a.personId;
10
11
```

Testcase Test Result

Output

firstName	lastName	city	state
Allen	Wang	null	null
Bob	Alice	New York City	New York

2. Employees Earning More Than Their Managers

Table: Employee

Column Name	Type
id	int
name	varchar
salary	int
managerId	int

id is the primary key column for this table.

Problem Summary

Write a solution to find the employees who earn more than their managers.

SQL Schema

```
CREATE TABLE Employee (id INT PRIMARY KEY,name VARCHAR(255),salary INT,managerId INT);
INSERT INTO Employee (id, name, salary, managerId) VALUES
(1, 'Joe', 70000, 3),
(2, 'Henry', 80000, 4),
(3, 'Sam', 60000, NULL),
(4, 'Max', 90000, NULL);
```

Employee

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	NULL
4	Max	90000	NULL

MySQL Solution



The screenshot shows a MySQL IDE interface. The top panel, titled 'Code', contains a MySQL query. The query is as follows:

```
1 # Write your MySQL query statement below
2 SELECT e.name AS Employee FROM Employee e
3 JOIN Employee m ON e.managerId = m.id
4 WHERE e.salary > m.salary;
```

The bottom panel, titled 'Testcase' and 'Test Result', shows the output of the query. The output is a table with two columns: 'Employee' and 'Joe'.

Employee
Joe

3. Customer Who Visited but Did Not Make Any Transactions

Table: Visits

Column Name	Type
-------------	------

visit_id	int
customer_id	int

visit_id is the primary key column for this table.

Table: Transactions

Column Name	Type
transaction_id	int
visit_id	int
amount	int

transaction_id is the primary key column for this table.

Problem Summary

Write a solution to find the IDs of the users who visited without making any transactions and the number of times they made these types of visits.

SQL Schema

Create table If Not Exists Visits(visit_id int, customer_id int)

Create table If Not Exists Transactions(transaction_id int, visit_id int, amount int)

Truncate table Visits insert into Visits (visit_id, customer_id) values ('1', '23')

insert into Visits (visit_id, customer_id) values ('2', '9') insert into Visits (visit_id,

customer_id) values ('4', '30') insert into Visits (visit_id, customer_id) values ('5',

'54') insert into Visits (visit_id, customer_id) values ('6', '96') insert into Visits

(visit_id, customer_id) values ('7', '54') insert into Visits (visit_id, customer_id)

values ('8', '54') Truncate table Transactions insert into Transactions

(transaction_id, visit_id, amount) values ('2', '5', '310') insert into Transactions

(transaction_id, visit_id, amount) values ('3', '5', '300') insert into Transactions

(transaction_id, visit_id, amount) values ('9', '5', '200') insert into Transactions

(transaction_id, visit_id, amount) values ('12', '1', '910') insert into Transactions

(transaction_id, visit_id, amount) values ('13', '2', '970') **Visits**

visit_id	customer_id
1	23
2	9
4	30
5	54
6	96
7	54
8	54

Transactions

transaction_id	visit_id	amount
2	5	310
3	5	300

9	5	200
12	1	910
13	2	970

MySQL Solution

The screenshot shows a MySQL IDE interface. The top pane displays a SQL query, and the bottom pane shows the test results.

```

1  # Write your MySQL query statement below
2  SELECT
3      v.customer_id,
4      COUNT(v.visit_id) AS count_no_trans
5  FROM
6      Visits v
7  LEFT JOIN
8      Transactions t ON v.visit_id = t.visit_id
9  WHERE
10     t.transaction_id IS NULL
11 GROUP BY
12     v.customer_id;
  
```

The test results pane shows the following output:

customer_id	count_no_trans
30	1
96	1
54	2

4. Rising Temperature

Table: Weather

Column Name	Type
Id	int
RecordDate	date
Temperature	int

id is the primary key column for this table.

Problem Summary

Write a solution to find all dates' id with higher temperatures compared to its previous dates (yesterday).

SQL Schema

Create table If Not Exists Weather (Id int, RecordDate date, Temperature int);
Truncate table Weather; insert into Weather (Id, RecordDate, Temperature)
values ('1', '2015-01-01', '10'); insert into Weather (Id, RecordDate, Temperature)
values ('2', '2015-01-02', '25'); insert into Weather (Id, RecordDate, Temperature)
values ('3', '2015-01-03', '20'); insert into Weather (Id, RecordDate, Temperature)
values ('4', '2015-01-04', '30');

Weather

Id	RecordDate	Temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

MySQL Solution

```
</> Code
MySQL Auto
1 # Write your MySQL query statement below
2 select w1.id
3 from Weather w1, Weather w2
4 where (w1.Temperature > w2.Temperature and
5        datediff(w1.recordDate, w2.recordDate) = 1)
```



5. Managers with at Least 5 Direct Reports

Table: Employee

Column Name	Type
Id	int
Name	varchar
Department	varchar
mangerId	int

Id is the primary key column for this table.

Problem Summary

Write a solution to find managers with at least **five direct reports**.

SQL Schema

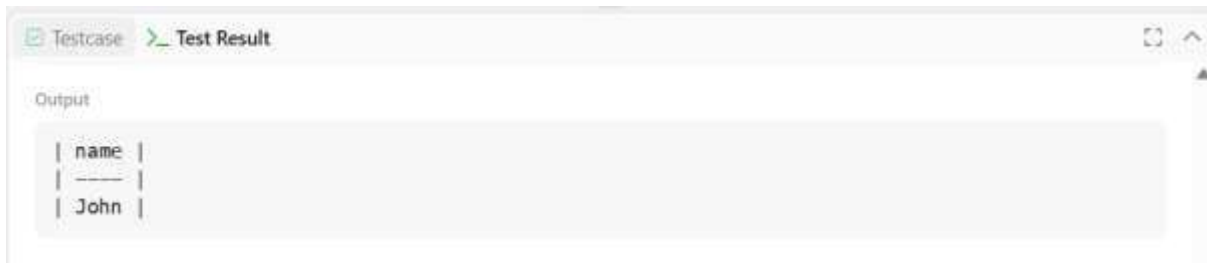
Create table If Not Exists Employee (id int, name varchar(255), department varchar(255), managerId int) Truncate table Employee insert into Employee (id, name, department, managerId) values ('101', 'John', 'A', NULL) insert into Employee (id, name, department, managerId) values ('102', 'Dan', 'A', '101') insert into Employee (id, name, department, managerId) values ('103', 'James', 'A', '101') insert into Employee (id, name, department, managerId) values ('104', 'Amy', 'A', '101') insert into Employee (id, name, department, managerId) values ('105', 'Anne', 'A', '101') insert into Employee (id, name, department, managerId) values ('106', 'Ron', 'B', '101')

Employee

id	name	department	managerId
101	John	A	NULL
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Ron	B	101

MySQL Solution

```
MySQL < Auto-
1. # Write your MySQL query statement below
2. SELECT name
3. FROM Employee
4. WHERE id IN
5.     (SELECT managerid
6.      FROM Employee
7.      GROUP BY managerid
8.      HAVING COUNT(id) >= 5);
```



6. Average Time of Process per Machine

Table: Activity

Column Name	Type
machine_id	int
process_id	int
activity_type	enum
timestamp	float

machine_id is the primary key column for this table.

Problem Summary

Write a solution to find the **average time** each machine takes to complete a process.

SQL Schema

```
Create table If Not Exists Activity (machine_id int, process_id int, activity_type
ENUM('start', 'end'), timestamp float) Truncate table Activity insert into
Activity (machine_id, process_id, activity_type, timestamp) values
('0', '0', 'start', '0.712'),
('0', '0', 'end', '1.52'),
('0', '1', 'start', '3.14'), ('0',
'1', 'end', '4.12'),
('1', '0', 'start', '0.55'), ('1',
'0', 'end', '1.55'),
('1', '1', 'start', '0.43'),
('1', '1', 'end', '1.42'),
('2', '0', 'start', '4.1'),
('2', '0', 'end', '4.512'),
('2', '1', 'start', '2.5'),
('2', '1', 'end', '5');
```

Activity

machine_id	process_id	activity_type	timestamp
0	0	start	0.712
0	0	end	1.52
0	1	start	3.14
0	1	end	4.12
1	0	start	0.55
1	0	end	1.55
1	1	start	0.43
1	1	end	1.42
2	0	start	4.1

2	0	end	4.512
2	1	start	2.5
2	1	end	5

MySQL Solution

The screenshot shows a MySQL IDE interface. The top pane, titled 'Code', contains a SQL query:


```
1 # Write your MySQL query statement below
2 select machine_id,
3        round(sum(if(activity_type='start', -timestamp, timestamp)) / count(distinct process_id), 3) as
4        processing_time
5 from Activity
6 group by machine_id
7 order by machine_id;
```

 The bottom pane, titled 'Test Result', shows the output of the query as a table:

machine_id	processing_time
0	0.894
1	0.995
2	1.456

7. Product Sales Analysis I

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

sale_id,year is the primary key column for this table.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key column for this table.

Problem Summary

Write a solution to report the product_name, year, and price for each sale_id in the Sales table.

SQL Schema

Create table Sales (sale_id int, product_id int, year int, quantity int, price int);
 Create table Product (product_id int, product_name varchar(10)); Truncate table Sales;
 insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');
 insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');
 insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');
 Truncate table Product; insert into Product (product_id, product_name) values ('100', 'Nokia');
 insert into Product (product_id, product_name) values ('200', 'Apple');
 insert into Product (product_id, product_name) values ('300', 'Samsung');

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product

product_id	product_name
100	Nokia
200	Apple
300	Samsung

MySQL Solution



The screenshot shows a MySQL IDE interface. At the top, there are tabs for 'Testcase' and 'Test Result'. Below the tabs, the 'Output' section displays a table with the following data:

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

Below the output, the 'Code' section shows a MySQL query:

```
1 # Write your MySQL query statement below
2 select p.product_name, s.year, s.price
3 from Sales s
4 left join Product p
5 on s.product_id = p.product_id
```

8. Product Sales Analysis II

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

sale_id,year is the primary key column for this table.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key column for this table.

Problem Summary

Write an SQL query that reports the total quantity sold for every product id.

SQL Schema

Create table Sales (sale_id int, product_id int, year int, quantity int, price int);
Create table Product (product_id int, product_name varchar(10)); Truncate table Sales;
insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');
insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');
insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');
Truncate table Product; insert into Product (product_id, product_name) values ('100', 'Nokia');
insert into Product (product_id, product_name) values ('200', 'Apple');
insert into Product (product_id, product_name) values ('300', 'Samsung');

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product

product_id	product_name
100	Nokia
200	Apple
300	Samsung

MySQL Solution

```
</> Code
MySQL v Auto
1 # Write your MySQL query statement below
2 select product_id, sum(quantity) as total_quantity
3 from Sales
4 group by product_id;
```

Testcase	Test Result
----------	-------------

Output

product_id	total_quantity
100	22
200	15

9. Students and Examinations Table:

Students

Column Name	Type
student_id	int
student_name	varchar

student_id is the primary key column for this table.

Table: Subjects

Column Name	Type
subject_name	varchar

subject_name is the primary key column for this table.

Table: Examinations

Column Name	Type
student_id	int
subject_name	varchar

Problem Summary

Write an SQL query to find the number of times each student attended each exam.
Order the result table by student_id and subject_name.

SQL Schema

```
Create table If Not Exists Students (student_id int, student_name varchar(20));
Create table If Not Exists Subjects (subject_name varchar(20));
Create table If Not Exists Examinations (student_id int, subject_name varchar(20));
Truncate table Students; insert into Students (student_id, student_name) values ('1', 'Alice'); insert into Students (student_id, student_name) values ('2', 'Bob'); insert into Students (student_id, student_name) values ('13', 'John'); insert into Students (student_id, student_name) values ('6', 'Alex'); Truncate table Subjects; insert into Subjects (subject_name) values ('Math'); insert into Subjects (subject_name) values ('Physics'); insert into Subjects (subject_name) values ('Programming'); Truncate table Examinations; insert into Examinations (student_id, subject_name) values ('1', 'Math'); insert into Examinations (student_id, subject_name) values ('1', 'Physics'); insert into Examinations (student_id, subject_name) values ('1', 'Programming'); insert into Examinations (student_id, subject_name) values ('2', 'Programming'); insert into Examinations (student_id, subject_name) values ('1', 'Physics'); insert into Examinations (student_id, subject_name) values ('1', 'Math'); insert into Examinations (student_id, subject_name) values ('13', 'Math'); insert into Examinations (student_id, subject_name) values ('13', 'Programming'); insert into Examinations (student_id, subject_name) values ('13', 'Physics'); insert into Examinations (student_id, subject_name) values ('2', 'Math'); insert into Examinations (student_id, subject_name) values ('1', 'Math');
```

Students

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

Subjects

subject_name
Math
Physics

Programming

Examination

student_id	subject_name
1	Math
1	Physics
1	Programming
2	Programming
1	Physics
1	Math
13	Math
13	Programming
13	Physics
2	Math

1	Math
---	------

MySQL Solution

Code
MySQL Auto

```

1 # Write your MySQL query statement below
2 select t.student_id, t.student_name, t.subject_name,
3        if(e.attended_exams is null, 0, e.attended_exams) as attended_exams
4 from
5     (select *
6      from Students
7      cross join Subjects) t
8 left join
9     (select *, count(*) as attended_exams
10      from Examinations
11      group by student_id, subject_name) e
12 on t.student_id = e.student_id and t.subject_name = e.subject_name
13 order by student_id, subject_name;

```

Testcase Test Result

Output

student_id	student_name	subject_name	attended_exams
1	Alice	Math	3
1	Alice	Physics	2
1	Alice	Programming	1
2	Bob	Math	1
2	Bob	Physics	0
2	Bob	Programming	1
6	Alex	Math	0
6	Alex	Physics	0
6	Alex	Programming	0
13	John	Math	1
13	John	Physics	1
13	John	Programming	1

View less

10. Department Highest Salary

Table: Employee

Column Name	Type
id	int
name	varchar
salary	int
departmentId	int

id is the primary key column for this table.

Table: Department

Column Name	Type
id	int
name	varchar

id is the primary key column for this table.

Problem Summary

Write a solution to find employees who have the highest salary in each of the departments.

SQL Schema

```
Create table If Not Exists Employee (id int, name varchar(255), salary int, departmentId int);
Create table If Not Exists Department (id int, name varchar(255)); Truncate table
Employee; insert into Employee (id, name, salary, departmentId) values ('1', 'Joe',
'70000', '1'); insert into Employee (id, name, salary, departmentId) values ('2', 'Jim',
'90000', '1'); insert into Employee (id, name, salary, departmentId) values ('3', 'Henry',
'80000', '2'); insert into Employee (id, name, salary, departmentId) values ('4', 'Sam',
'60000', '2'); insert into Employee (id, name, salary, departmentId) values ('5', 'Max',
'90000', '1'); Truncate table Department; insert into Department (id, name) values ('1',
'IT'); insert into Department (id, name) values ('2', 'Sales');
```

Employee

id	name	salary	departmentId
1	Joe	70000	1
2	Jim	90000	1
3	Henry	80000	2
4	Sam	60000	2
5	Max	90000	1

Department

id	name
1	IT
2	Sales

MySQL Solution

Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 WITH cteRowNum AS (
3     SELECT DepartmentId, name employee, Salary,
4         DENSE_RANK() OVER ( PARTITION BY DepartmentId ORDER BY Salary desc ) my_rank
5     FROM Employee
6 )
7 SELECT d.name Department, Employee, Salary
8     FROM cteRowNum, Department d
9     WHERE departmentid = d.id
10    and my_rank = 1;
```

Testcase Test Result

Output

Department	Employee	Salary
IT	Jim	90000
IT	Max	90000
Sales	Henry	80000